

# PaddleFormers 贡献模型示例

目录

- [引言](#)
- [0 贡献前准备](#)
- [PaddleFormers环境配置](#)
- [2 开发流程](#)
- [2.1 创建模型](#)
  - [2.1.1 定义模型配置类（参考代码链接）](#)
  - [2.1.2 定义模型类](#)
    - [Paddle TorchAPI 映射表：](#)
    - [Ernie4\\_5PretrainedModel](#)
    - [Ernie4\\_5Model](#)
    - [Ernie4\\_5ForCausalLM](#)
    - [Ernie4\\_5ForCausalLMPipe](#)
    - [CheckList](#)
  - [2.1.3 定义分词器](#)
- [2.2 Auto注册](#)
- [2.3 Lazy import 导包控制](#)
- [2.4 模型下载源映射](#)
- [2.5 模型动态图单卡推理验证](#)
- [2.6 模型生成验证](#)
- [2.7 模型训练验证](#)
- [2.8 模型单测](#)
- [2.9 模型template验证](#)

## 引言

PaddleFormers属于飞桨开源的高性能大模型套件，为研究人员和开发者提供了丰富的预训练模型库和便捷的二次开发接口。本文将详细介绍如何向PaddleFormers社区贡献新模型的完整流程，帮助开发者规范、高效地完成模型提交。本文以ERNIE-4.5-0.3B模型为例，提供贡献模型教程。

## 0 贡献前准备

PaddleFormers中，新增一个模型工作流程与目前主流的Transformers是比较类似的。**Transformers** 是一个用于最先进机器学习模型的**模型定义框架**，支持文本、计算机视觉、音频、视频和多模态模型。

**Transformers**的源码目录结构：Transformers关于模型组网部分的代码存放于src/transformers目录下。

| 关于模型部分代码目录         |                                                                                |  |
|--------------------|--------------------------------------------------------------------------------|--|
| src/transformers   |                                                                                |  |
| -----commands      | Transformers 的CLI实现代码，目前支持chat、convert、download、run等功能                         |  |
| -----data          | 用于模型指标评价的数据预处理                                                                 |  |
| -----distributed   | 关于分布式并行的代码设计，暂时只有一个Config                                                      |  |
| -----generation    | 大模型自回归生成的模块，包括采样算法、Logits概率分布后处理、流方式输出等                                        |  |
| -----intergrations | 集成其他库的功能，例如accelerate、deepspeed、PyTorch的张量并行、FlashAttention等                   |  |
| -----kernels       | 集成CUDA实现的算子实现代码                                                                |  |
| -----loss          | 损失函数，包括语言建模、图像分类、目标检测等任务                                                       |  |
| -----models        | 存放模型组网源代码                                                                      |  |
| -----onnx          | 导出onnx模型的代码                                                                    |  |
| -----pipelines     | 语言、音频、视觉中任务封装成了一个简单的API，只需传递输入，就能获取到有意义的输出                                     |  |
| -----quantizers    | 量化模型，支持多种量化算法 (https://huggingface.co/docs/transformers/quantization/overview) |  |
| -----sagemaker     | Transformers部署到AWS中的兼容性设计                                                      |  |
| -----utils         | Dummy 类的实现与设计，导入Transformers不依赖Torch                                           |  |
| -----py文件          | 目录下包括很多其他py文件，涵盖KVCache封装、激活值封装、trainer等功能                                     |  |

图1 transformers核心代码目录

在 **Transformers** 框架中新增模型结构时，需要在 *models* 目录下进行相应的代码添加。*models* 目录的组织结构如图2所示，其中包含 *auto* 文件夹以及各类模型实现的文件夹（每个模型均对应一个独立的文件夹，这个文件夹的名字是与配置文件中model\_type对应）。

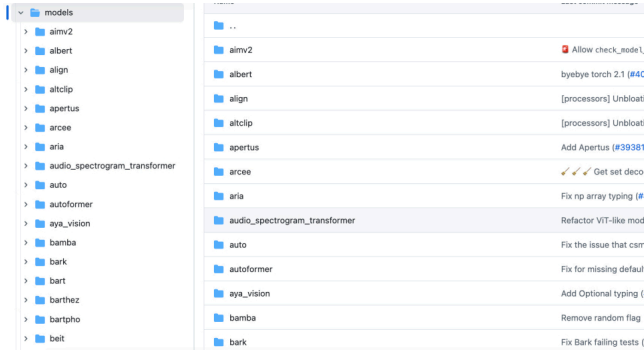


图2 models目录下的组织结构

**Transformers的auto模块：**auto模块支持实例化自动选择合适的类，例如AutoModelForCausalLM 可以根据模型路径、名字来自动实例化。详细可以参考（[https://huggingface.co/docs/transformers/v4.56.1/en/model\\_doc/auto#auto-classes](https://huggingface.co/docs/transformers/v4.56.1/en/model_doc/auto#auto-classes)）

```
</> Python |
1 from transformers import AutoModelForCausalLM
2 from transformers import Ernie4_5ForCausalLM
3
4 # 实例化Ernie4.5自回归语言模型
5 AutoModelForCausalLM.from_pretrained("baidu/ERNIE-4.5-0.3B-PT") 等价于
   Ernie4_5ForCausalLM.from_pretrained("baidu/ERNIE-4.5-0.3B-PT")
```

**Transformers中Ernie4.5模型目录结构组织：**

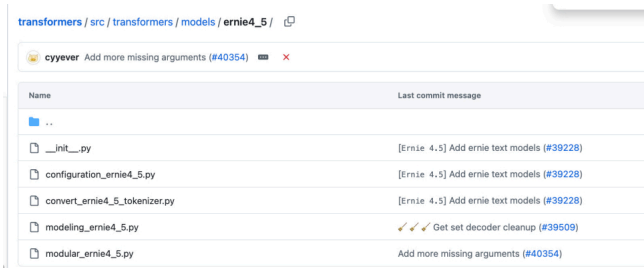


图3 transformers中ernie4.5目录结构

| 1 | 文件名                           | 文件含义                                                   | 是否迁移到PaddleFormers |
|---|-------------------------------|--------------------------------------------------------|--------------------|
| 2 | configuration_ernie4_5.py     | 里面的Ernie4_5Config类，定义Ernie4.5模型的超参数                    | ✓                  |
| 3 | convert_ernie4_5_tokenizer.py | 从Paddle的Tokenizer配置转化成HF的Tokenizer配置文件                 | ✗                  |
| 4 | modeling_ernie4_5.py          | 定义模型组网代码                                               | ✓                  |
| 5 | modular_ernie4_5.py           | 减少代码维护成本的modular设计，能够快速从已有transformers组网生成新的modeling文件 | ✗                  |

表1 models文件夹下文件列表

在transformers 中构建一个模型组网需要实现以下几个基础类：

| 1 | 父类                  | 子类                                |                                                                                                                                                                                                                                                                 |
|---|---------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | PretrainedConfig    | Ernie4_5Config                    | 实例化后的config对象可以用于指定模型属性，例如网络层数、注意力头数或词汇表大小等。                                                                                                                                                                                                                    |
| 3 | PreTrainedModel     | Ernie4_5Model、Ernie4_5ForCausalLM | 根据配置文件定义的模型架构，需要至少定义两个类。<br>1.以Ernie4_5为例，首先定义Ernie4_5Model模型仅返回特征hidden states，<br>2. 为特定任务工作的模型需要另外定义一个模型名+For+Task的类，从而将Ernie4_5ForCausalLM的隐藏状态输出转换为有意义的结果<br>• 自回归语言建模任务定义Ernie4_5ForCausalLM（预测下一个token的概率）<br>• 序列分类定义Ernie4_5ForSequenceClassification。 |
| 4 | PreTrainedTokenizer | -                                 | 用于NLP任务模型的文本预处理<br>• 把文本字符串 → token → token ID（数字）<br>• 负责 分词、特殊符号处理（[CLS]、[SEP]）、padding/truncation、batch 编码<br>• Ernie4.5模型使用的是LlamaTokenizer                                                                                                                 |
|   | ProcessorMixin      | -                                 | 多模态任务的统一的预处理器                                                                                                                                                                                                                                                   |

|   |                          |   |                                                                                                                                                                                                                                                                                                                                           |
|---|--------------------------|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 |                          |   | <ul style="list-style-type: none"><li>把多个模态的处理器 (tokenizer、image processor、feature extractor) 封装成一个整体</li><li>例如：<ul style="list-style-type: none"><li>Qwen2VLProcessor = Qwen2Tokenizer + Qwen2VLImageProcessor</li><li>Qwen2_5OmniProcessor = Qwen2_5Tokenizer + Qwen2_5VLImageProcessor + Wav2Vec2FeatureExtractor</li></ul></li></ul> |
| 6 | BaseImageProcessor       | - | <b>视觉任务的预处理器</b> <ul style="list-style-type: none"><li>图像 → 标准化后的张量 (pixel values)</li><li>包括 resize、crop、归一化、channel 排列 (HWC → CHW) 等操作</li><li>例如：<ul style="list-style-type: none"><li>ViTImageProcessor</li></ul></li></ul>                                                                                                           |
| 7 | Wav2Vec2FeatureExtractor | - | <b>音频任务</b> <ul style="list-style-type: none"><li>读取 原始 <b>waveform</b> (浮点数组, 通常是 <b>16kHz</b> 采样率的 <b>PCM</b> 信号)</li><li>做 归一化、重采样、<b>padding</b>、分帧 等预处理</li><li>例如：<ul style="list-style-type: none"><li>Wav2Vec2FeatureExtractor</li></ul></li></ul>                                                                                |

表2 不同类型模型需要实现的基础类

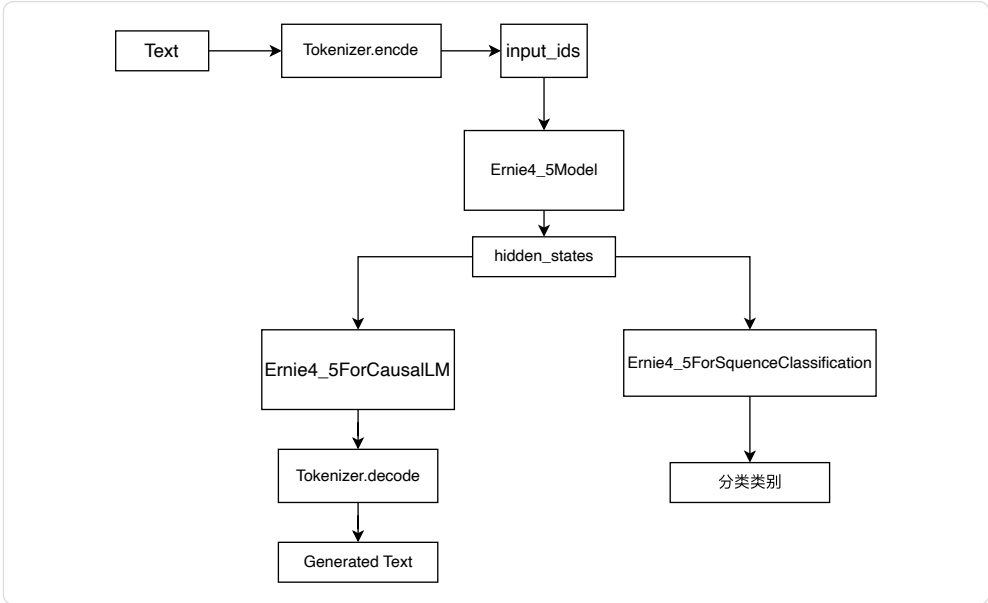


图4 Ernie4.5Model 和具有不同任务头下的Ernie4\_5ForCasualLM和Ernie4\_5ForSequenceClassification

Checkpoint仓库文件列表：Transformers中支持的模型会将模型权重托管在huggingface中，Ernie-4.5-0.3B-PT包含以下文件列表

|   |                        |                                                |
|---|------------------------|------------------------------------------------|
| 1 | 文件                     | 说明                                             |
| 2 | added_tokens.json      | 存放额外的特殊字符字典, (用于SlowTokenizer)                 |
| 3 | chat_template.jinja    | 使用jinja语法                                      |
| 4 | config.json            | 包含该尺寸组网的配置                                     |
| 5 | generation_config.json | 用于自回归生成的超参数                                    |
| 6 | model.safetensors      | 存储网络的实际权重                                      |
| 7 | tokenizer.json         | 存储分词器和词表的信息 (FastTokenizer使用)                  |
| 8 | tokenizer.model        | SlowTokenizer读取使用的 SentencePiece 分词器           |
| 9 | tokenizer_config.json  | Tokenizer的配置信息, 使用哪个Tokenizer类会放在这里, 特殊字符的处理规则 |

表3 权重仓库中文件列表说明

Transformers支持SlowTokenizer跟FastTokenizer两种，详细可查看  
([https://huggingface.co/docs/transformers/v4.56.1/en/main\\_classes/tokenizer#tokenizer](https://huggingface.co/docs/transformers/v4.56.1/en/main_classes/tokenizer#tokenizer))  
Transformers支持权重分片存储的safetensors ([https://huggingface.co/docs/transformers/main/en/big\\_models](https://huggingface.co/docs/transformers/main/en/big_models))

|                                  |                 |                                          |              |
|----------------------------------|-----------------|------------------------------------------|--------------|
| model-00118-of-00123.safetensors | 4.90 GB [10 MB] | Add files using upload-large-folder tool | 3 months ago |
| model-00119-of-00123.safetensors | 4.90 GB [10 MB] | Add files using upload-large-folder tool | 3 months ago |
| model-00120-of-00123.safetensors | 4.90 GB [10 MB] | Add files using upload-large-folder tool | 3 months ago |
| model-00121-of-00123.safetensors | 4.90 GB [10 MB] | Add files using upload-large-folder tool | 3 months ago |
| model-00122-of-00123.safetensors | 4.90 GB [10 MB] | Add files using upload-large-folder tool | 2 months ago |
| model-00123-of-00123.safetensors | 5.00 GB [10 MB] | Add files using upload-large-folder tool | 3 months ago |
| model.safetensors.index.json     | 960 KB [8 B]    | Add files using upload-large-folder tool | 3 months ago |

图5 分片存储，model.safetensors.index.json，存放了具体的信息

## 1. PaddleFormers环境配置

代码仓：<https://github.com/PaddlePaddle/PaddleFormers>

- 推荐直接使用官方提供的 Docker 容器，容器中已经拉取了 PaddleFormers 仓库并完成了安装：

```
</>
1 # 以cuda12.6为例
2 docker run --gpus all --name paddleformers-work -v $(pwd):/work \
3   -w=/work --shm-size=512G --network=host -it \
4   ccr-2vdh3abv-pub.cnc.bj.baidubce.com/paddlepaddle/paddle:3.3.0-gpu-cuda12.6-cudnn9.5 /bin/bash
5
6 # cuda12.9镜像: ccr-2vdh3abv-pub.cnc.bj.baidubce.com/paddlepaddle/paddle:3.3.0-gpu-cuda12.9-cudnn9.9
7 # cuda13.0镜像: ccr-2vdh3abv-pub.cnc.bj.baidubce.com/paddlepaddle/paddle:3.3.0-gpu-cuda13.0-cudnn9.13
```

- 如果不想使用官方镜像，也可以拉取源码安装：

```
</>
1 git clone https://github.com/PaddlePaddle/PaddleFormers.git
2 cd PaddleFormers
3 # cuda12.6
4 python -m pip install -e '.[paddlefleet]' --extra-index-url https://www.paddlepaddle.org.cn/packages/stable/cu126/
5 # cuda12.9
6 # python -m pip install -e '.[paddlefleet]' --extra-index-url https://www.paddlepaddle.org.cn/packages/stable/cu129/
7 # cuda13.0
8 # python -m pip install -e '.[paddlefleet]' --extra-index-url https://www.paddlepaddle.org.cn/packages/stable/cu130/
```

## 2 开发流程

### 2.1 创建模型

每一个模型结构都会有一个唯一的简称（model\_type），我们ERNIE-4.5-0.3B在PaddleFormers中简称为ernie4\_5。在paddleformers/transformers新建模型简称为ernie4\_5的目录，目录中存放模型定义、模型配置、分词器（可选）等。

- configuration.py文件存放模型配置
- modeling.py存放组网定义代码
- tokenizer.py 存放分词器相关代码

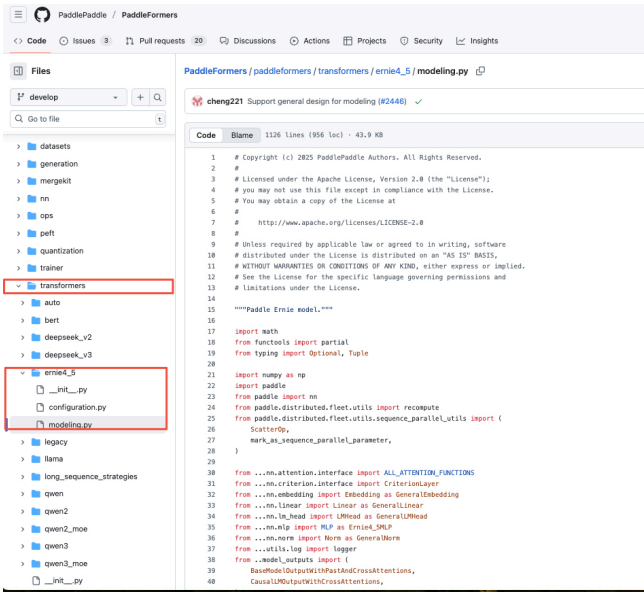


图6 模型存放位置

#### 2.1.1 定义模型配置类（参考代码链接）

在ernie4\_5目录下新增 configuration.py文件，所有的模型配置都继承于paddleformers.transformers.configuration\_utils.PretrainedConfig，

</> Python |

1 2 3 4 5

1. 首先导入PretrainedConfig: `from ..configuration_utils import PretrainedConfig`

2. 然后定义模型配置类: `class Ernie4_5Config(PretrainedConfig)`,

\* 模型配置类中包含模型需要用到的超参数。

\* 将需要传入参数写在初始化方法 `def __init__`的参数列表中，并设置为属性

\* 设置 `model_type` 类变量，相同`model_type`模型有对应的`XXXConfig`

```
""" Ernie4_5 model configuration."""
from ..configuration_utils import PretrainedConfig

class Ernie4_5Config(PretrainedConfig):
    """
    Configuration class for Ernie4_5 model.

    This class stores the configuration of an Ernie4_5 model, defining the model architecture.
    It inherits from PretrainedConfig and can be used to control model outputs.
    """

    model_type = "ernie4_5"

    def __init__(
        self,
        vocab_size=32000,
        hidden_size=768,
        intermediate_size=11008,
        max_position_embeddings=32768,
        num_hidden_layers=2,
        num_attention_heads=2,
        head_dim=128,
        scale_qk_coeff=1.0,
        initializer_range=0.02,
        rms_norm_eps=1e-6,
        use_cache=False,
        use_flash_attention=False,
        recompute=False,
        recompute_granularity="core_attn",
        recompute_use_reentrant=False,
        tie_word_embeddings=True,
        use_rmsnorm=True,
        pad_token_id=0,
        bos_token_id=1,
        eos_token_id=2,
        use_bias=False,
        rope_theta=10000,
        fuse_rope=False,
        fuse_softmax_mask=False,
        fuse_linear=False,
        max_sequence_length=None,
        ignored_index=-100,
        attention_dropout_prob=0.0,
        hidden_act="silu",
        hidden_dropout_prob=0.0,
        num_key_value_heads=None,
        micro_batch_size=-1,
```

图7 Ernie4\_5Config设置

需要配置的参数主要如表1所示，Config类需要保证以下规范。

- 在迁移过程中，Config中的变量命名尽可能与原huggingface组网保持一致
- 新增PaddleFormers组网需要额外用到的变量
- 利用self.register\_unsavable\_keys方法去减少不必要保存的变量，在保存config时（config.save\_pretrained()）无需存储到config.json的变量
- PaddleFormers和transformers PretrainedConfig字段对比 [Transformers与PaddleFormers 的PretrainedConfig 字段对比](#)

|    | 参数名                                  | 类型    | 含义                              | 说明 |
|----|--------------------------------------|-------|---------------------------------|----|
| 1  |                                      |       |                                 |    |
| 2  | <code>vocab_size</code>              | int   | 词汇表大小，即模型能识别的不同token的数量         |    |
| 3  | <code>hidden_size</code>             | int   | 表示Word Embedding 和隐藏层特征维度       |    |
| 4  | <code>intermediate_size</code>       | int   | FFN（前馈网络）中间层的维度，通常比hidden_size大 |    |
| 5  | <code>max_position_embeddings</code> | int   | 模型支持的最大序列长度（位置编码的最大位置）          |    |
| 6  | <code>num_hidden_layers</code>       | int   | 大模型的Decoder网络层数                 |    |
| 7  | <code>num_attention_heads</code>     | int   | 注意力的头数                          |    |
| 8  | <code>head_dim</code>                | int   | 注意力每一个头，对应的特征纬度                 |    |
| 9  | <code>scale_qk_coeff</code>          | float | 注意力计算中，Q和K的缩放系数                 |    |
| 10 | <code>initializer_range</code>       | float | 权重初始化的范围（标准差）                   |    |
| 11 | <code>rms_norm_eps</code>            | float | RMSNorm中epsilon值，保障数值稳定         |    |
| 12 | <code>use_cache</code>               | bool  | 推理过程是否使用KVCache缓存加速自回归生成        |    |
| 13 | <code>recompute</code>               | bool  | 是否开启重新计算                        |    |
|    | <code>recompute_granularity</code>   | str   | recompute的细粒度                   |    |

|    |  |                        |       |                                  |                                                                          |
|----|--|------------------------|-------|----------------------------------|--------------------------------------------------------------------------|
| 14 |  | pad_token_id           | int   | 用于填充的token id                    |                                                                          |
| 15 |  | bos_token_id           | int   | 句子开始前添加的token id                 |                                                                          |
| 16 |  | eos_token_id           | int   | 句子结束后添加的token id                 |                                                                          |
| 17 |  | fuse_swiglu            | bool  | 是否使用融合的Swish-GLU激活函数             |                                                                          |
| 18 |  | fuse_linear            | bool  | RoPe融合线性层操作                      |                                                                          |
| 19 |  | max_sequence_length    | int   | 实际使用的最大序列长度                      |                                                                          |
| 20 |  | ignored_index          | int   | 计算loss时需要忽略的token id             |                                                                          |
| 21 |  | add_tail_layers        | int   | 在尾部添加的Empty层，在流水线并行训练策略会用到       |                                                                          |
| 22 |  | attention_dropout_prob | float | 注意力层的dropout概率                   |                                                                          |
| 23 |  | hidden_act             | str   | MLP 中 hidden states 的激活函数，默认silu |                                                                          |
| 24 |  | hidden_dropout_prob    | float | hidden states dropout 的概率        |                                                                          |
| 25 |  | num_key_value_heads    | int   | 分组注意力算法，KV的注意力头数                 |                                                                          |
| 26 |  | micro_batch_size       | int   |                                  | 后续将删去                                                                    |
| 27 |  | pp_seg_method          | str   | 使用流水线并行训练策略时，组网分段方法              | 后续将删去，<br>dpo_config跟<br>kto_config已经在<br>父类，<br>pp_seg_method<br>也将移到父类 |
| 28 |  | dpo_config             | -     | DPO训练的配置                         |                                                                          |
| 29 |  | kto_config             | -     | KTO训练的配置                         |                                                                          |

表2 配置文件主要参数列表

如果需要新增通用训练配置字段(如“moe\_subbatch\_token\_num”), 这类字段通过yaml文件控制，有特殊训练需求时添加。例如：`moe_subbatch_token_num: 512`

需要遵循以下规则(或参考PR#2684), 其中是否需要新增、字段归类与xxx\_attributes命名等问题可以找 @郑露静

1.添加至configuration.py-LlmMetaConfig

</>

Python

2.添加至configuration.py-PretrainedConfig

</>

Python

2.1.2 定义模型类

与Transformers类似，我们的组网结构定义在modeling.py文件下。对于一个大语言模型，需要至少定义四个模型 **Ernie4\_5PretrainedModel**，**Ernie4\_5Model**（基模型），**Ernie4\_5ForCausalLM**（自回归生成文本）， **Ernie4\_5ForCausalLMPipe**(流水线并行组网)。

Paddle TorchAPI 映射表：

发现对API映射有问题可详询高翔

[https://www.paddlepaddle.org.cn/documentation/docs/zh/develop/guides/model\\_convert/convert\\_from\\_pytorch/pytorch\\_api\\_mapping\\_cn.html](https://www.paddlepaddle.org.cn/documentation/docs/zh/develop/guides/model_convert/convert_from_pytorch/pytorch_api_mapping_cn.html)

Ernie4\_5PretrainedModel

所有的模型类的父类都会包括PretrainedModel类（paddleformers.transformers.model\_utils.PretrainedModel）， **PretrainedModel**类提供了模型加载(**from\_pretrained()**)和保存(**save\_pretrained()**)的功能。

定义class Ernie4\_5PretrainedModel 的过程中，需要完成的工作如下：

设置类变量：config\_class、base\_model\_prefix、transpose\_weight\_keys。

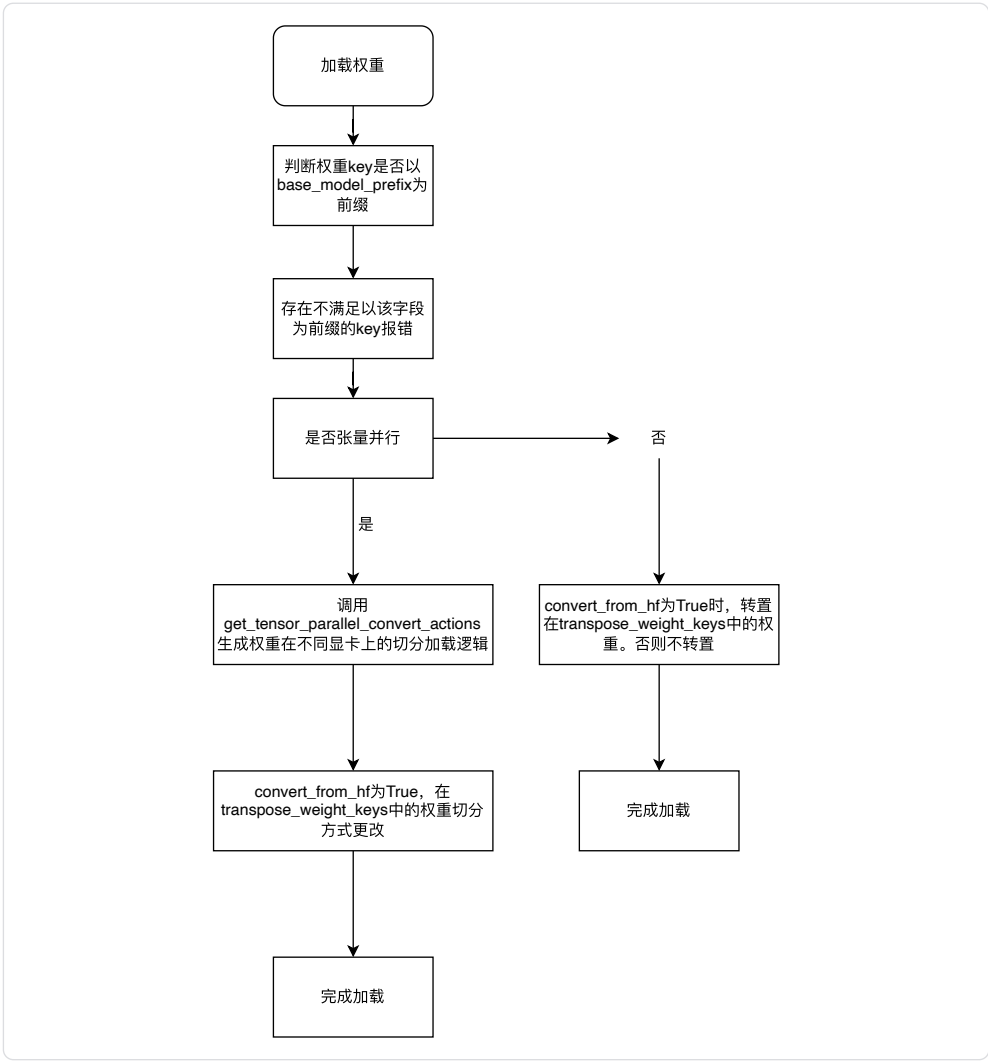
|   |                       |                                  |
|---|-----------------------|----------------------------------|
| 1 | 变量                    | 说明                               |
| 2 | config_class          | 在加载权重过程中用于获取模型的config            |
| 3 | base_model_prefix     | 在加载权重时，与权重的state_dict 的key进行前缀判断 |
|   | transpose_weight_keys | 指明需转置的权重，规则：                     |

|   |                                                                                                                                                          |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4 | <div>1. Paddle与PyTorch的Linear层权重存储维度不一致，需要将xxx.weight转置。</div> <div>2. transpose_weight_keys是一个列表，如果里面一个元素key满足以key.weight结尾，则会转置。（目前暂时不支持正则匹配key）</div> |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------|

</> Ernie4.5 示例 Python

```
1 config_class = Ernie4_5Config , 设置为我们刚定义好的Config。
2 base_model_prefix = "model" , 与模型参数关键字的前缀有关，建议统一model命名。
3 transpose_weight_keys = ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"], 适配加载&保存HF-torch模型，设置需要对linear weight权重转置的key
```

• 重写\_get\_tensor\_parallel\_mappings方法：该方法影响模型在张量并行时加载和保存权重的策略。在unified\_checkpoint这个加载和存储下会用到base\_model\_prefix。下面以加载权重为例，进行说明



\_get\_tensor\_parallel\_mappings:

- 功能：实现了权重key与切分方式的映射关系，权重竖向切分还是横向切分这个功能。
- 注意点：每一个权重key要保证能映射到对应的切分规则。
  - 假如我们有两个model.embed\_tokens.weight和model.layers.46.embed\_tokens.weight，如果我们匹配规则的字典中有{"embed\_tokens.weight":partial(fn, is\_column=False)，此时model.layers.46.embed\_tokens.weight 权重切分规则会覆盖掉model.embed\_tokens.weight的权重切分规则，导致model.embed\_tokens.weight无法被切分。
  - 务必保证每个需要张量并行key都有映射到正确的切分规则！

</> Ernie4.5 \_get\_tensor\_parallel\_mappings写法示例 Python

```
1 @classmethod
2 def _get_tensor_parallel_mappings(cls, config, is_split=True):
```

```

3      """Generate tensor parallel mappings for model conversion."""
4      from ..conversion_utils import split_or_merge_func
5
6      fn = split_or_merge_func(
7          is_split=is_split,
8          tensor_parallel_degree=config.tensor_parallel_degree,
9          tensor_parallel_rank=config.tensor_parallel_rank,
10         num_attention_heads=config.num_attention_heads,
11     )
12
13     LAYER_COLWISE = [
14         "self_attn.q_proj.weight",
15         "self_attn.k_proj.weight",
16         "self_attn.v_proj.weight",
17         "mlp.up_proj.weight",
18         "mlp.gate_proj.weight",
19     ]
20
21     LAYER_ROWWISE = ["self_attn.o_proj.weight", "mlp.down_proj.weight"]
22
23     BIAS_KEYS = [
24         "self_attn.q_proj.bias",
25         "self_attn.k_proj.bias",
26         "self_attn.v_proj.bias",
27         "mlp.gate_proj.bias",
28         "mlp.up_proj.bias",
29         "self_attn.o_proj.bias",
30         "mlp.down_proj.bias",
31         "lm_head.bias",
32     ]
33
34     def make_base_actions():
35         actions = {
36             "lm_head.weight": partial(fn, is_column=False),
37             f"{cls.base_model_prefix}.embed_tokens.weight": partial(fn, is_column=False),
38         }
39         for layer_idx in range(config.num_hidden_layers):
40             actions.update(
41                 {
42                     f"{cls.base_model_prefix}.layers.{layer_idx}.{k}": partial(fn, is_column=True)
43                     for k in LAYER_COLWISE
44                 }
45             )
46             actions.update(
47                 {
48                     f"{cls.base_model_prefix}.layers.{layer_idx}.{k}": partial(fn, is_column=False)
49                     for k in LAYER_ROWWISE
50                 }
51             )
52             # bias
53             if config.use_bias:
54                 actions.update(
55                     {f"{cls.base_model_prefix}.layers.0.{b}": partial(fn, is_column=True) for b in BIAS_KEYS}
56                 )
57
58         return actions
59
60     mappings = make_base_actions()
61     return mappings

```

## Ernie4\_5Model

**Ernie4\_5Model**定义了大模型组网整体结构，包含词嵌入编码、解码器、norm 三部分。



```
@register_base_model
class Ernie4_5Model(Ernie4_5PretrainedModel):
    """The core ERNIE transformer model"""

    def __init__(self, config: Ernie4_5Config):
        """Initialize the ERNIE model architecture.

        Args:
            config (Ernie4_5Config): Model configuration.
        """
        super().__init__(config)
        self.padding_idx = config.pad_token_id
        self.vocab_size = config.vocab_size
        self.hidden_size = config.hidden_size
        self.config = config
        self.embed_tokens = GeneralEmbedding.create(
            config=config, num_embeddings=config.vocab_size, embedding_dim=config.hidden_size
        )
        self.layers = nn.LayerList([Ernie4_5DecoderLayer(config, i) for i in range(config.num_hidden_layers)])
        self.norm = GeneralNorm.create(
            config=config,
            norm_type="rms_norm",
            hidden_size=config.hidden_size,
            has_bias=config.use_bias,
            norm_eps=self.config.rms_norm_eps,
        )
```

图8 Ernie4\_5Model定义部分

在定义各个模块过程中，我们需要使用paddleformers.nn下面通用组件进行模型组网的定义。映射关系如表2所示,其开发设计文档在：[PaddleFormers通用类抽象设计文档](#)

|   |                                                       |                                                         |                                        |
|---|-------------------------------------------------------|---------------------------------------------------------|----------------------------------------|
| 1 | PyTorch模块                                             | PaddleFormers组件                                         | 额外说明                                   |
| 2 | torch.nn.Embedding                                    | paddleformers.nn.embedding.Embedding                    |                                        |
| 3 | torch.nn.Linear                                       | paddleformers.nn.linear.Linear                          |                                        |
| 4 | RMSNorm、torch.nn.LayerNorm                            | paddleformers.nn.norm.Norm                              | <b>PaddleFormers中的Norm组件需要应对输入并行情况</b> |
| 5 | LMHead                                                | paddleformers.nn.lm_head.LMHead                         |                                        |
| 6 | MLP                                                   | paddleformers.nn.mlp.MLP                                |                                        |
| 7 | self.loss_function (transformers)                     | paddleformers.nn.criterion.interface.CriterionLayer     |                                        |
| 8 | AttentionInterface (transformers中attention_forward封装) | paddleformers.nn.criterion.interface.AttentionInterface |                                        |

表2 Torch模块与paddleformers组件映射关系

- DynamicCache组件：PaddleFormers 目前使用和 transformers 对齐的 DynamicCache 组件。
  - 使用时需要导入 `from ..cache_utils import Cache, DynamicCache`
  - 需要在 xxModel, xxAttention类中对 DynamicCache 进行操作。Model 负责 DynamicCache 的初始化，Attention 中对 DynamicCache 对象进行更新。

```
1 paddleformers
2
3 </> Qwen3Model-paddleformers Python
4
5 1 if use_cache and past_key_values is None:
6   past_key_values = DynamicCache(config=self.config)
7 2 cache_length = past_key_values.get_seq_length() if past_key_values is not None else 0
8
9 3 if position_ids is None:
10  position_ids = paddle.arange(seq_length, dtype="int64").expand((batch_size, seq_length))
11
12
13 </> Qwen3Attention-paddleformers Python
14
15 1 cos, sin = position_embeddings
16 2 query_states, key_states = apply_rotary_pos_emb(query_states, key_states, cos, sin)
17 3
18 4 # [bs, num_head, seq_len, head_dim]
19 5 if past_key_values is not None:
20  key_states, value_states = past_key_values.update(key_states, value_states, self.layer_idx)
21 6
22 7
23 8 attention_interface = ALL_ATTENTION_FUNCTIONS[self.config._attn_implementation]
```

- RotaryEmbedding组件：该实现基本与transformers一致，可以参考Qwen2/3组网的实现，其中configuration.py已支持 `rope_config_validation` 检查。

**NOTE:** RotaryEmbedding的forward函数，在paddle.amp.auto\_cast上下文中使用cast会导致cuda graph不兼容

</> Qwen2/3RotaryEmbedding实现

Plain Text

```
1 from ..modeling_rope_utils import ROPE_INIT_FUNCTIONS, dynamic_rope_update
2
3 class Qwen3RotaryEmbedding(nn.Layer):
4     def __init__(self, config: Qwen3Config):
5         super().__init__()
6         self.max_seq_len_cached = config.max_position_embeddings
7         self.original_max_seq_len = config.max_position_embeddings
8         self.config = config
9         rope_parameters = self.config.rope_parameters
10        self.rope_type = rope_parameters.get("rope_type", rope_parameters.get("type", "default"))
11        rope_init_fn = self.compute_default_rope_parameters
12        if self.rope_type != "default":
13            rope_init_fn = ROPE_INIT_FUNCTIONS[self.rope_type]
14        inv_freq, self.attention_scaling = rope_init_fn(self.config)
15
16        self.register_buffer("inv_freq", inv_freq, persistable=False)
17        self.original_inv_freq = inv_freq
18
19    @staticmethod
20    def compute_default_rope_parameters(
21        config: Optional[Qwen3Config] = None,
22        seq_len: Optional[int] = None,
23    ) -> tuple["paddle.Tensor", float]:
24        """
25        Computes the inverse frequencies according to the original RoPE implementation
26        Args:
27            config ([`PreTrainedConfig`]):
28                The model configuration.
29            seq_len (`int`, *optional*):
30                The current sequence length. Unused for this type of RoPE.
31        Returns:
32            Tuple of (`paddle.Tensor`, `float`), containing the inverse frequencies for the RoPE embeddings and the
33            post-processing scaling factor applied to the computed cos/sin (unused in this type of RoPE).
34        """
35        base = config.rope_parameters["rope_theta"]
36        dim = getattr(config, "head_dim", None) or config.hidden_size // config.num_attention_heads
37
38        attention_factor = 1.0 # Unused in this type of RoPE
39
40        # Compute the inverse frequencies
41        inv_freq = 1.0 / (base ** (paddle.arange(0, dim, 2, dtype=paddle.int64).astype(dtype=paddle.float32) / dim))
42        return inv_freq, attention_factor
43
44    @dynamic_rope_update
45    def forward(self, x, position_ids):
46        with paddle.amp.auto_cast(enable=False):
47            inv_freq_expanded = self.inv_freq[None, :, None].float().expand([position_ids.shape[0], -1, 1])
48
49            position_ids_expanded = position_ids[:, None, :].float()
50
51            freqs = (inv_freq_expanded.float() @ position_ids_expanded.float()).transpose(1, 2)
52
53            emb = paddle.concat((freqs, freqs), axis=-1)
54
55            cos = emb.cos() * self.attention_scaling
```

```

56         sin = emb.sin() * self.attention_scaling
57
58         return cos.to(dtype=x.dtype), sin.to(dtype=x.dtype)
59     ...
60
61 class Qwen3Model(Qwen3PretrainedModel):
62     def __init__(self, config: Qwen3Config):
63         super().__init__(config)
64         ...
65         self.rotary_emb = Qwen3RotaryEmbedding(config=config)
66         ...
67     def forward(
68         self,
69         input_ids: paddle.Tensor = None,
70         ...
71         position_ids: Optional[paddle.Tensor] = None,
72         ...
73     ):
74         ...
75         if position_ids is None:
76             position_ids = paddle.arange(seq_length, dtype="int64").expand((batch_size, seq_length))
77         ...
78         # create position embeddings to be shared across the decoder layers
79         position_embeddings = self.rotary_emb(hidden_states, position_ids)
80
81

```

- `apply_rotary_emb`方法：实现分交错RoPE(GLM系列)和标准RoPE(Qwen系列)，实现基本与transformers一致

</> `apply_rotary_emb`实现

Python

```

1 def rotate_half(x):
2     """Rotates half the hidden dims of the input."""
3     x1 = x[..., : x.shape[-1] // 2]
4     x2 = x[..., x.shape[-1] // 2 :]
5     return paddle.cat([-x2, x1], axis=-1)
6
7 # 标准RoPE
8 def apply_rotary_pos_emb(q, k, cos, sin, position_ids=None, unsqueeze_dim=1):
9     """Applies Rotary Position Embedding to the query and key tensors."""
10    cos = cos.unsqueeze(unsqueeze_dim)
11    sin = sin.unsqueeze(unsqueeze_dim)
12    q_embed = (q * cos) + (rotate_half(q) * sin)
13    k_embed = (k * cos) + (rotate_half(k) * sin)
14    return q_embed.astype(q.dtype), k_embed.astype(k.dtype)
15
16 # 交错RoPE
17 def apply_rotary_pos_emb(q, k, cos, sin, position_ids=None, unsqueeze_dim=1):
18     """Applies Rotary Position Embedding to the query and key tensors."""
19    cos = cos.unsqueeze(unsqueeze_dim)
20    sin = sin.unsqueeze(unsqueeze_dim)
21
22    # Keep half or full tensor for later concatenation
23    rotary_dim = cos.shape[-1]
24    q_rot, q_pass = q[..., :rotary_dim], q[..., rotary_dim:]
25    k_rot, k_pass = k[..., :rotary_dim], k[..., rotary_dim:]
26
27    # Apply rotary embeddings on the first half or full tensor
28    q_embed = (q_rot * cos) + (rotate_half(q_rot) * sin)
29    k_embed = (k_rot * cos) + (rotate_half(k_rot) * sin)
30
31    # Concatenate back to full shape
32    q_embed = paddle.cat([q_embed, q_pass], axis=-1)
33    k_embed = paddle.cat([k_embed, k_pass], axis=-1)
34

```

```

35     return q_embed, k_embed
36
37
38 class <YourModel>Attention(nn.Layer):
39     def __init__(self, config, layer_idx=0):
40         ...
41     def forward(
42         self,
43         hidden_states,
44         ...
45     ):
46         ...
47         cos, sin = position_embeddings
48         query_states, key_states = apply_rotary_pos_emb(query_states, key_states, cos, sin)

```

完成组件定义后，必须实现方法forward和recompute\_training（通过config.recompute字段控制）。

```

</> Python |
1  @paddle.jit.not_to_static
2  def recompute_training(
3      self,
4      layer_module,
5      hidden_states,
6      attention_mask,
7      attn_mask_startend_row_indices,
8      position_ids,
9      output_attentions,
10     past_key_value,
11     use_cache,
12 ):
13     def create_custom_forward(module):
14         def custom_forward(*inputs):
15             return module(*inputs)
16
17         return custom_forward
18
19     hidden_states = recompute(
20         create_custom_forward(layer_module),
21         hidden_states,
22         attention_mask,
23         attn_mask_startend_row_indices,
24         position_ids,
25         output_attentions,
26         past_key_value,
27         use_cache,
28     )
29     return hidden_states

```

PaddleFormers / paddleformers / transformers / ernie4\_5 / modeling.py

| Code | Blame | 1126 lines (956 loc) · 43.9 KB |
|------|-------|--------------------------------|
|------|-------|--------------------------------|

```
782     def forward(
783     ):
784         causal_attention_mask = None
785
786         # decoder layers
787         all_hidden_states = () if output_hidden_states else None
788         all_self_attns = () if output_attentions else None
789         next_decoder_cache = () if use_cache else None
790
791         for idx, (decoder_layer) in enumerate(self.layers):
792             if output_hidden_states:
793                 all_hidden_states += (hidden_states,)
794
795             past_key_value = past_key_values[idx] if past_key_values is not None else None
796             has_gradient = not hidden_states.stop_gradient
797             if self.config.recompute and self.config.recompute_granularity == "full" and has_gradient:
798                 layer_outputs = self.recompute_training(
799                     decoder_layer,
800                     hidden_states,
801                     causal_attention_mask,
802                     attn_mask_start_row_indices,
803                     position_ids,
804                     output_attentions,
805                     past_key_value,
806                     use_cache,
807                 )
808             else:
809                 layer_outputs = decoder_layer(
810                     hidden_states,
811                     causal_attention_mask,
812                     attn_mask_start_row_indices,
813                     position_ids,
814                     output_attentions,
815                     past_key_value,
816                     use_cache,
817                 )
818             hidden_states, attns, key_values = layer_outputs
819             hidden_states = hidden_states.to(torch.float32)
820             key_values = key_values.to(torch.float32)
821             past_key_value = key_values
822             if output_attentions:
823                 all_self_attns += (attns,)
824
825         hidden_states = self.norm(hidden_states)
826         if output_hidden_states:
827             all_hidden_states += (hidden_states,)
828         next_decoder_cache = next_decoder_cache.to(torch.float32)
829         return hidden_states, all_self_attns, all_hidden_states, next_decoder_cache
```

图9 Ernie4\_5Model forward逻辑

- 配置causal mask
  - 复用configuration.py文件，其中 `self.layer_types` 处理逻辑根据模型进行修改，如Qwen2/3、GPT-OSS处理方式存在diff

</>

Python

- 修改原attention\_mask处理逻辑和decoder\_layer传入mask，区分“full\_attention”和“sliding\_attention”两种mask

</>

Python

```
1 from ..masking_utils import create_causal_masks_and_row_indices
2
3 # if attn_mask_startend_row_indices is not None:
4 #     attention_mask = None
5 # else:
6 #     attention_mask = self._prepare_decoder_attention_mask(
7 #         attention_mask, (batch_size, seq_length), cache_length, inputs_embeds.dtype
8 #     )
9
10 # attention_mask/attn_mask_startend_row_indices修改
11 # 考虑多个causal mask
12 mask_kwargs = {
13     "config": self.config,
14     "inputs_embeds": inputs_embeds,
15     "batch_size": batch_size,
16     "seq_length": seq_length,
17     "cache_length": cache_length,
18     "attention_mask": attention_mask,
19     "attn_mask_startend_row_indices": attn_mask_startend_row_indices,
20     "prepare_decoder_attention_mask": self._prepare_decoder_attention_mask,
21 }
22 causal_mask_mapping, attn_mask_startend_row_indices_mapping = create_causal_masks_and_row_indices(**mask_kwargs)
23
24 # 只使用单个causal mask
25 mask_kwargs = {
26     "config": self.config,
27     "inputs_embeds": inputs_embeds,
28     "batch_size": batch_size,
29     "seq_length": seq_length,
30     "cache_length": cache_length,
31     "attention_mask": attention_mask,
32     "attn_mask_startend_row_indices": attn_mask_startend_row_indices,
33     "prepare_decoder_attention_mask": self._prepare_decoder_attention_mask,
34     "return_mapping": False,
35 }
36 causal_mask, attn_mask_startend_row_indices = create_causal_masks_and_row_indices(**mask_kwargs)
37
38 # decoder_layer修改
```

```

39 layer_outputs = decoder_layer(
40     hidden_states,
41     causal_mask_mapping[decoder_layer.attention_type],
42     # causal_mask,
43     position_ids,
44     past_key_value,
45     use_cache,
46     position_embeddings,
47     attn_mask_startend_row_indices=attn_mask_startend_row_indices_mapping[
48         decoder_layer.attention_type
49     ],
50     # attn_mask_startend_row_indices,
51     batch_size=batch_size,
52 )

```

**Ernie4\_5DecoderLayer类定义：**如下代码块所示，需要注意的点

- 继续定义Ernie4\_5Attention、Ernie4\_5MLP（可以从paddleformers.nn.mlp.MLP）两个类
- Norm层定义：**定义input\_layernorm 和 post\_attention\_layernorm，导入GeneralNorm。如果norm的输入是并行在不同GPU，在定义时需要合理传入input\_is\_parallel=True。
  - 张量并行切分的输入，补充入参input\_is\_parallel=config.tensor\_parallel\_degree>1
  - 序列并行切分的输入，补充input\_is\_parallel=config.sequence\_parallel

```

</> Plain Text |
1     self.input_layernorm = GeneralNorm.create(
2         config=config,
3         norm_type="rms_norm",
4         hidden_size=config.hidden_size,
5         has_bias=config.use_bias,
6         norm_eps=self.config.rms_norm_eps,
7         input_is_parallel=config.sequence_parallel,
8     )

```

- 返回规范：**实现forward时的返回值为元组（hidden\_states, self\_attn\_weights（可选），present\_key\_value（可选））
- 参数规范：**hidden\_states、attention\_mask、attn\_mask\_startend\_row\_indices、position\_ids、position\_embeddings、past\_key\_value等输入参数的名称保持一致，否则会导致部分参数在流水线并行模块中传递失败

```

</> Python |
1 class Ernie4_5DecoderLayer(nn.Layer):
2     """A single transformer decoder layer in ERNIE model.
3
4     Contains self-attention and feed-forward components,
5     support, residual connections, and layer normalization.
6     """
7
8     def __init__(self, config, layer_idx):
9         """Initialize the decoder layer.
10
11         Args:
12             config (Ernie4_5Config): Model configuration.
13             layer_idx (int): Index of this layer in the transformer stack
14         """
15         super().__init__()
16         self.hidden_size = config.hidden_size
17         self.layer_idx = layer_idx
18         self.config = config
19         self.self_attn = Ernie4_5Attention(config, layer_idx)
20         self.mlp = Ernie4_5MLP(config)
21         self.input_layernorm = GeneralNorm.create(
22             config=config,
23             norm_type="rms_norm",
24             hidden_size=config.hidden_size,
25             has_bias=config.use_bias,

```

```

26         norm_eps=self.config.rms_norm_eps,
27         input_is_parallel=config.sequence_parallel,
28     )
29     self.post_attention_layernorm = GeneralNorm.create(
30         config=config,
31         norm_type="rms_norm",
32         hidden_size=config.hidden_size,
33         has_bias=config.use_bias,
34         norm_eps=self.config.rms_norm_eps,
35         input_is_parallel=config.sequence_parallel,
36     )
37
38     self.hidden_dropout = nn.Dropout(p=config.hidden_dropout_prob, mode="upscale_in_train")
39
40     if config.sequence_parallel:
41         self.post_attention_layernorm.enable_sequence_parallel()
42         if not hasattr(config, "disable_ffn_model_parallel"):
43             self.input_layernorm.enable_sequence_parallel()
44             if config.use_bias:
45                 mark_as_sequence_parallel_parameter(self.self_attn.o_proj.bias)
46                 mark_as_sequence_parallel_parameter(self.mlp.down_proj.bias)
47
48     def forward(
49         self,
50         hidden_states: paddle.Tensor,
51         attention_mask: Optional[paddle.Tensor] = None,
52         attn_mask_startend_row_indices: Optional[paddle.Tensor] = None,
53         position_ids: Optional[paddle.Tensor] = None,
54         position_embeddings: Optional[paddle.Tensor] = None,
55         output_attentions: Optional[bool] = False,
56         past_key_value: Optional[Tuple[paddle.Tensor]] = None,
57         use_cache: Optional[bool] = False,
58     ) -> Tuple[paddle.Tensor, Optional[Tuple[paddle.Tensor, paddle.Tensor]]]:
59         """Forward pass through the decoder layer.
60
61         Args:
62             hidden_states (paddle.Tensor): Input tensor [batch_size, seq_len, hidden_size]
63             attention_mask (Optional[paddle.Tensor]): Attention mask tensor
64             attn_mask_startend_row_indices (Optional[paddle.Tensor]): Indices for variable length attention
65             position_ids (Optional[paddle.Tensor]): Position indices for rotary embeddings
66             position_embeddings (Optional[paddle.Tensor]): Position embeddings tensor
67             output_attentions (Optional[bool]): Whether to return attention weights
68             past_key_value (Optional[Tuple[paddle.Tensor]]): Cached key/value states
69             use_cache (Optional[bool]): Whether to cache key/value states
70
71         Returns:
72             Union: Various output combinations depending on arguments:
73                 - Base case: Hidden states tensor
74                 - With attention: Tuple of (hidden_states, attention_weights)
75                 - With cache: Tuple of (hidden_states, cached_key_value)
76         """
77         residual = hidden_states
78
79         hidden_states = self.input_layernorm(hidden_states)
80
81         # Self Attention
82         hidden_states, self_attn_weights, present_key_value = self.self_attn(
83             hidden_states=hidden_states,
84             past_key_value=past_key_value,
85             attention_mask=attention_mask,
86             attn_mask_startend_row_indices=attn_mask_startend_row_indices,
87             position_embeddings=position_embeddings,
88             output_attentions=output_attentions,
89             use_cache=use_cache,
90         )

```

```

91
92     with model_parallel_dropout(self.config):
93         hidden_states = self.hidden_dropout(hidden_states) + residual
94
95     # Fully Connected
96     residual = hidden_states
97     hidden_states = self.post_attention_layernorm(hidden_states)
98     hidden_states = self.mlp(hidden_states)
99
100    with model_parallel_dropout(self.config):
101        hidden_states = self.hidden_dropout(hidden_states) + residual
102
103    outputs = (hidden_states,)
104
105    if output_attentions:
106        outputs += (self.attn_weights,)
107
108    if use_cache:
109        outputs += (present_key_value,)
110
111    # remove empty tuple for pipeline parallel
112    if type(outputs) is tuple and len(outputs) == 1:
113        outputs = outputs[0]
114    return outputs
115

```

实现注意力类Ernie4\_5Attention需要关注变量属性定义、**attention\_forward**配置、**KV Cache**实现三个部分

- 变量属性定义

- q\_proj、k\_proj、v\_proj定义：指定tp\_plan为colwise切分
- o\_proj 定义：指定tp\_plan为rowwise切分
- attn\_implementation定义：config.attn\_implementation
- qkv的张量并行：与attention\_head 数量有关， attention\_head需要被tp并行度整除

```

</> Python |
1 self.q_proj = GeneralLinear.create(
2     self.hidden_size,
3     q_hidden_size,
4     has_bias=config.use_bias,
5     config=config,
6     fuse_matmul_bias=config.fuse_linear,
7     tp_plan="colwise",
8 )
9 self.k_proj = GeneralLinear.create(
10     self.hidden_size,
11     kv_hidden_size,
12     has_bias=config.use_bias,
13     config=config,
14     fuse_matmul_bias=config.fuse_linear,
15     tp_plan="colwise",
16 )
17 self.v_proj = GeneralLinear.create(
18     self.hidden_size,
19     kv_hidden_size,
20     has_bias=config.use_bias,
21     config=config,
22     fuse_matmul_bias=config.fuse_linear,
23     tp_plan="colwise",
24 )
25 self.o_proj = GeneralLinear.create(
26     q_hidden_size,
27     self.hidden_size,
28     has_bias=config.use_bias,
29     config=config,

```



```

30     fuse_matmul_bias=config.fuse_linear,
31     tp_plan="rowwise",
32 )
33 self.attn_implementation = config._attn_implementation

```

- **attention\_forward配置**：配置方案与Transformers相似，额外新增attn\_mask\_startend\_row\_indices等参数。返回的attn\_output维度为B,L,D。

```

</> Python |
1 if attn_mask_start_row_indices is None and attention_mask is None:
2     self.attn_implementation = "sdpa"
3 attention_interface = ALL_ATTENTION_FUNCTIONS[self.attn_implementation]
4 attn_output, attn_weights = attention_interface(
5     self,
6     query=query_states,
7     key=key_states,
8     value=value_states,
9     attention_mask=attention_mask,
10    attn_mask_startend_row_indices=attn_mask_startend_row_indices,
11    dropout=self.config.get("attention_dropout_prob", 0.0) if self.training else 0.0,
12    scaling=self.scaling,
13 )

```

- **KV Cache实现（用于推理）**：past\_key\_values是一个元组，在Ernie4\_5Model forward函数中进行初始化。在启动use\_cache时  
past\_key\_values = tuple([None] \* len(self.layers))
  - past\_key\_values：元组长度为 Decoder 层数、元组中每一个元素类型是列表，列表中包含两个Tensor，分别是第 i 层 注意力Key和Value缓存，Tensor的维度是b,l,h,d
  - 推理过程中，注意力计算前会从past\_key\_values中获取该token之前所有序列的Key Value 缓存，与Key、Value拼接。同时将拼接后的值更新 past\_key\_value
- **Moe 模块注意事项 (tp+sp)**：
  - 适用范围：带moe模块的模型(如glm4moe, qwen2moe等) 接入sp策略
  - 操作：接入sequence parallel时，注意不要对moe模块中的linear应用sp策略，应在创建moe模块的linear前把config.sequence\_parallel赋值  
为False
  - 原因：由于sp会把长度按照tp数切分，而moe模块每个expert分到的token长度不一致，导致forward会hang住。因此不能切分。参考2735
  - 如何验证：tp+sp策略下，构建model后print(model)，检查expert Linear创建的类型是ColumnParallelLinear（没开sp仅开tp，正确）还是  
ColumnSequenceParallelLinear(开了sp，错误)

## Ernie4\_5ForCausalLM

Ernie4\_5ForCausalLM 继承于Ernie4\_5PretrainedModel，是用于自回归建模的模型，在这个类实现中：

- 类变量的添加：\_keys\_to\_ignore\_on\_load\_missing = ["lm\_head.weight"]
- 定义Decoder组网为：self.model = Ernie4\_5Model(config)
- 定义lm\_head部分：self.lm\_head = GeneralLMHead(config)
- 定义criterion部分：self.criterion = CriterionLayer(config)
- 调用 self.tie\_weights() # 处理词嵌入embedding 和 LMHead 参数复用时的情况
- 实现模型forward部分
  - 组网调用
  - 适配损失函数
- 适配自回归推理
  - 可选：如果对attention mask有特殊处理，重写prepare\_attention\_mask\_for\_generation

```

</> Python |
1 def prepare_attention_mask_for_generation(self, input_ids, pad_token_id, eos_token_id):
2     """Avoid using attention_mask with flash_attn on generation."""
3     if self.config.use_flash_attention:
4         return None
5     return super().prepare_attention_mask_for_generation(input_ids, pad_token_id, eos_token_id)

```

- 可选：重写prepare\_inputs\_for\_generation，将kwargs中的变量传入到模型forward中

```

</> Python |

```

```

1  def prepare_inputs_for_generation(
2      self,
3      input_ids,
4      use_cache=False,
5      past_key_values=None,
6      inputs_embeds=None,
7      **kwargs,
8  ):
9      if past_key_values:
10         input_ids = input_ids[:, -1:]
11
12         attention_mask = kwargs.get("attention_mask", None)
13
14         # if `inputs_embeds` are passed, we only want to use them in the 1st generation step
15         if inputs_embeds is not None and past_key_values is None:
16             model_inputs = {"inputs_embeds": inputs_embeds}
17         else:
18             model_inputs = {"input_ids": input_ids}
19
20         model_inputs.update(
21             {
22                 "past_key_values": past_key_values,
23                 "use_cache": True, # use_cache,
24                 "attention_mask": attention_mask,
25                 "return_dict": True,
26             }
27         )
28
29     return model_inputs

```

- 可选：重写update\_model\_kwargs\_for\_generation：在生成下一个token时，attention\_mask等需要动态更新，需要在这个方法实现对下一次模型forward输入的更新

```

</> Python |
1  @staticmethod
2  def update_model_kwargs_for_generation(outputs, model_kwargs, is_encoder_decoder=False):
3      # update cache
4      if isinstance(outputs, tuple) and len(outputs) > 1 and not isinstance(outputs[1], paddle.Tensor):
5          model_kwargs["past_key_values"] = outputs[1]
6
7      if isinstance(outputs, CausalLMOutputWithCrossAttentions) and "past_key_values" in outputs:
8          model_kwargs["past_key_values"] = outputs.past_key_values
9
10     # update token_type_ids with last value
11     if "token_type_ids" in model_kwargs and model_kwargs["token_type_ids"] is not None:
12         token_type_ids = model_kwargs["token_type_ids"]
13         model_kwargs["token_type_ids"] = paddle.concat([token_type_ids, token_type_ids[:, -1:]], axis=-1)
14     if not is_encoder_decoder and model_kwargs.get("attention_mask", None) is not None:
15         # update attention mask
16         attention_mask = model_kwargs["attention_mask"]
17         model_kwargs["attention_mask"] = paddle.concat(
18             [
19                 attention_mask,
20                 paddle.ones([attention_mask.shape[0], 1], dtype=attention_mask.dtype),
21             ],
22             axis=-1,
23         )
24     # update role_ids
25     if "role_ids" in model_kwargs and model_kwargs["role_ids"] is not None:
26         role_ids = model_kwargs["role_ids"]
27         model_kwargs["role_ids"] = paddle.concat([role_ids, role_ids[:, -1:]], axis=-1)
28
29     return model_kwargs

```

使用组网组件完成定义，分布式程序运行以下脚本，可以根据attention的heads数目调整mp\_degree。

```
</> test_tp.py Python |
1 from paddle.distributed import fleet
2 import paddle.distributed as dist
3 from paddleformers.transformers.ernie4_5.configuration import Ernie4_5Config
4 from paddleformers.transformers.ernie4_5.modeling import Ernie4_5ForCausalLM
5
6 def main():
7     strategy = fleet.DistributedStrategy()
8     strategy.hybrid_configs = {
9         "dp_degree": 1, # Data parallelism degree
10        "mp_degree": dist.get_world_size(), # Model parallelism degree (to be determined or set)
11        "pp_degree": 1, # Pipeline parallelism degree
12        "sharding_degree": 1, # Sharding parallelism degree
13    }
14    fleet.init(is_collective=True, strategy=strategy) # Initialize Fleet
15    hcg = fleet.get_hybrid_communicate_group() # Get the hybrid communicate group after initialization
16
17    # Get tensor parallel degree and rank after Fleet initialization
18    tensor_parallel_degree = hcg.get_model_parallel_world_size()
19    tensor_parallel_rank = hcg.get_model_parallel_rank()
20
21
22    model_path = "baidu/ERNIE-4.5-0.3B-PT"
23    config = Ernie4_5Config.from_pretrained(model_path)
24    config.tensor_parallel_degree = tensor_parallel_degree
25    config.tensor_parallel_rank = tensor_parallel_rank
26    model = Ernie4_5ForCausalLM.from_pretrained(model_path, config=config, convert_from_hf=True)
27    print(model)
28
29 if __name__ == "__main__":
30     main()
31 # python -u -m paddle.distributed.launch --devices "0,1" --nnodes=1 test_tp.py
```

执行命令后模型正常打印，模型正常加载，能看到ColumnParallelLinear等算子，说明TP成功接入

```
(up_proj): ColumnParallelLinear()
(down_proj): RowParallelLinear()
)
(input_layernorm): RMSNorm()
(post_attention_layernorm): RMSNorm()
(hidden_dropout): Dropout(p=0.0, axis=None, mode=upscale_in_train)
)
(17): Ernie4_5DecoderLayer(
  (self_attn): Ernie4_5Attention(
    (q_proj): ColumnParallelLinear()
    (k_proj): ColumnParallelLinear()
    (v_proj): ColumnParallelLinear()
    (o_proj): RowParallelLinear()
  )
  (mlp): MLP(
    (act_fn): Silu(inplace=False)
    (gate_proj): ColumnParallelLinear()
    (up_proj): ColumnParallelLinear()
    (down_proj): RowParallelLinear()
  )
  (input_layernorm): RMSNorm()
  (post_attention_layernorm): RMSNorm()
  (hidden_dropout): Dropout(p=0.0, axis=None, mode=upscale_in_train)
)
)
(norm): RMSNorm()
(rotary_emb): Ernie4_5RotaryEmbedding()
)
(lm_head): LMHead(hidden_size=1024, vocab_size=51712, dtype=paddle.bfloat16, vocab_parallel=True)
(criterion): CriterionLayer()
(loss_func): ParallelCrossEntropy()
)
```

图10 TP组网打印的输出

## Ernie4\_5ForCausalLMPipe

构建流水线并行模型，在modeling文件中定义类Ernie4\_5ForCausalLMPipe

- Ernie4\_5ForCausalLMPipe继承于paddleformers.nn.pp\_model.GeneralModelForCausalLMPipe
- 修改Ernie4\_5ForCausalLMPipe类变量，
- 如果新模型的embedding、position\_embedding计算方法与GeneralModelForCausalLMPipe给的默认方法不同，可以通过\_embed\_cls、\_rotary\_emb\_cls传入

```

1 class Ernie4_5ForCausalLMPipe(GeneralModelForCausalLMPipe):
2     config_class = Ernie4_5Config
3     _embed_cls = None
4     _rotary_emb_cls = None
5     _decoder_layer_cls = Ernie4_5DecoderLayer
6     _get_tensor_parallel_mappings = Ernie4_5Model._get_tensor_parallel_mappings
7     _init_weights = Ernie4_5Model._init_weights
8     _keep_in_fp32_modules = Ernie4_5Model._keep_in_fp32_modules
9     _tied_weights_keys = ["lm_head.weight"]
10    transpose_weight_keys = Ernie4_5Model.transpose_weight_keys
11
12
13 __all__ = ["Ernie4_5Model", "Ernie4_5ForCausalLM", "Ernie4_5ForCausalLMPipe"]

```

## CheckList

1.现在模型大多数参数类型为bfloat16，但存在部分参数为float32（例如moe中的gate，correction\_bias等），为了保证FP32参数被正确加载切不被amp cast为BF16，需要添加以下两行代码：

- Ernie4\_5\_MoePretrainedModel 类中加入\_keep\_in\_fp32\_modules，例如\_keep\_in\_fp32\_modules = ["mlp.gate.weight", "e\_score\_correction\_bias"]
- 如果此类中包含float32的参数，例如Glm4MoeTopkRouter 中包含self.weight类型为float32，那么需要在\_\_init\_\_中添加 self.\_cast\_to\_low\_precision = False

### 2.1.3 定义分词器

PaddleFormers分词器目前实现是基于Transformers中的Tokenizer进行包装实现，实现需要导入transformers、从paddleformers.transformers.tokenizer\_utils import warp\_tokenizer

```

</> tokenizer.py
Python |

1 import transformers as hf
2
3 from ..tokenizer_utils import warp_tokenizer
4
5 Ernie4_5Tokenizer = warp_tokenizer(hf.LlamaTokenizer)

```

## 2.2 Auto注册

AutoConfig、AutoTokenizer、AutoTokenizer等可以根据模型权重和配置，自动推断出config、tokenizer、Model应该从哪个类进行实例化、

AutoConfig（[添加位置](#)）：新增 ("ernie4\_5", "Ernie4\_5Config")

AutoTokenizer（[添加位置](#)）：新增("ernie4\_5", "Ernie4\_5Tokenizer"),

AutoModel([添加位置](#))：新增 ("Ernie4\_5", "ernie4\_5")

Ernie4\_5为组网中Ernie4\_5Model，Model前的模型名，ernie4\_5 为paddleformers.transformers下的目录名

## 2.3 Lazy import 导包控制

在两处位置新增import 控制逻辑

- paddleformers/transformers/ernie4\_5/\_\_init\_\_.py
  - [https://github.com/PaddlePaddle/PaddleFormers/blob/develop/paddleformers/transformers/ernie4\\_5/\\_\\_init\\_\\_.py](https://github.com/PaddlePaddle/PaddleFormers/blob/develop/paddleformers/transformers/ernie4_5/__init__.py)

```

import sys
from typing import TYPE_CHECKING

from ...utils.lazy_import import _LazyModule

import_structure = {
    "tokenizer": ["copyfile", "Ernie4_5Tokenizer"],
    "tokenizer_utils": ["PretrainedTokenizer"],
    "configuration": ["Ernie4_5Config"],
    "modeling": ["Ernie4_5Model", "Ernie4_5ForCausalLM"],
}

if TYPE_CHECKING:
    from .configuration import *
    from .modeling import *
else:
    sys.modules[__name__] = _LazyModule(
        __name__,
        globals()[ "__file__ "],
        import_structure,
        module_spec=__spec__,
    )

```

图12 lazy import 示例

- paddleformers/transformers/\_\_init\_\_.py
  - [https://github.com/PaddlePaddle/PaddleFormers/blob/develop/paddleformers/transformers/\\_\\_init\\_\\_.py#L170](https://github.com/PaddlePaddle/PaddleFormers/blob/develop/paddleformers/transformers/__init__.py#L170)
  - [https://github.com/PaddlePaddle/PaddleFormers/blob/develop/paddleformers/transformers/\\_\\_init\\_\\_.py#L388C1-L389C1](https://github.com/PaddlePaddle/PaddleFormers/blob/develop/paddleformers/transformers/__init__.py#L388C1-L389C1)

## 2.4 模型下载源映射

**i** 星河社区HF高热模型映射表: <https://ku.baidu-int.com/workbench/dst4wz1wSH3cBkMr3j/viwxrp46TS4Yf>

新增组网需同步添加/更新Huggingface、modelscope和星河社区三个下载源映射表，参考代码如下：

[https://github.com/PaddlePaddle/PaddleFormers/blob/develop/paddleformers/transformers/download\\_utils.py](https://github.com/PaddlePaddle/PaddleFormers/blob/develop/paddleformers/transformers/download_utils.py)

星河社区添加新增组网权重需联系aistudio同学或 @郑露静

```

</> download_utils.py添加模型映射 Python |

1 # qwen2
2 register_model_group(
3     models={
4         "Qwen2-0.5B": {
5             DownloadSource.HUGGINGFACE: "Qwen/Qwen2-0.5B",      # huggingface
6             DownloadSource.AISTUDIO: "ModelHub/Qwen2-0.5B",      # aistudio
7             DownloadSource.MODELScope: "Qwen/Qwen2-0.5B",        # modelscope
8         },
9         "Qwen2-1.5B": {
10             DownloadSource.HUGGINGFACE: "Qwen/Qwen2-1.5B",
11             DownloadSource.AISTUDIO: "ModelHub/Qwen2-1.5B",
12             DownloadSource.MODELScope: "Qwen/Qwen2-1.5B",
13         },
14         ...
15     }
16 )
17
18 # new model name
19 register_model_group(
20     models={
21         "<repo_name>": {
22             DownloadSource.HUGGINGFACE: "<namespace/repo_name>",
23             DownloadSource.AISTUDIO: "<namespace/repo_name>",      # 星河社区目前统一以“ModelHub”作namespace
24             DownloadSource.MODELScope: "<namespace/repo_name>",
25         },
26     }
27 )

```

## 2.5 模型动态图单卡推理验证

```
</> Python |  
  
1 from paddleformers.transformers import AutoConfig, AutoModelForCausalLM, AutoTokenizer  
2  
3 # 方法1. 加载Paddle模型  
4 model_path = "checkpoint/ERNIE-4.5-0.3B-Paddle"  
5 config = AutoConfig.from_pretrained(model_path)  
6 tokenizer = AutoTokenizer.from_pretrained(model_path)  
7 model = AutoModelForCausalLM.from_pretrained(model_path)  
8  
9 # 方法2: 如果需要加载HF-torch模型, 新增一个参数convert_from_hf, 参数默认是None  
10 model_path = "baidu/ERNIE-4.5-0.3B-PT"  
11 model = AutoModelForCausalLM.from_pretrained(  
12     model_path,  
13     convert_from_hf=True,  
14 )  
15  
16 inputs = tokenizer.encode("Hello world", return_tensors='pd')  
17 inputs.pop("attention_mask", None)  
18  
19 outputs = model.generate(**inputs,max_new_tokens=128, return_dict=True)  
20 # print(outputs)  
21 print(tokenizer.decode(outputs[0][0].tolist()))  
22  
23 # 模型保存  
24 # 如果需要将模型保存为适配HF-torch模型, 新增一个参数save_to_hf  
25 model.save_pretrained(save_dir=save_dir, save_to_hf=True)  
26
```

单卡组网精度: 需要保证跟原组网前20个Token的logits精度差距在1e-2内,

推荐环境:

- paddleformers 1.0
- paddle 3.3
- transformers 4.57.6
- torch 2.9.1

参考代码如下:

```
</> Python |  
  
1 from transformers import AutoTokenizer, AutoModelForCausalLM  
2 from paddleformers.transformers import Glm4MoeForCausalLM, Qwen3ForCausalLM  
3 import torch  
4 import paddle  
5  
6 model_path = "/work/Qwen3-0.6B-Base"  
7  
8 tokenizer = AutoTokenizer.from_pretrained(model_path)  
9  
10 input_ids = tokenizer("where is Beijing")["input_ids"]  
11  
12 pt_model = AutoModelForCausalLM.from_pretrained(model_path, dtype="bfloat16", _attn_implementation="eager").cuda()  
13 pd_model = Qwen3ForCausalLM.from_pretrained(model_path, convert_from_hf=1, _attn_implementation="eager", dtype="bfloat16")  
14  
15 pt_model.eval()  
16 pd_model.eval()  
17 pt_output = pt_model(input_ids=torch.tensor([input_ids]).cuda())  
18 pd_output = pd_model(input_ids=paddle.tensor([input_ids]))  
19 print("torch output", pt_output)  
20 print("paddleformers output", pd_output)  
21 print("diff:", abs(pt_output.logits.float().detach().cpu().numpy() - pd_output[0].float().cpu().numpy()).mean())
```

## 2.6 模型生成验证

PaddleFormers 文本生成代码参考：

```
</> Bash |  
  
1 from paddleformers.transformers import AutoTokenizer, AutoModelForCausalLM  
2  
3 tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen3-0.6B-Base")  
4 model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen3-0.6B-Base", dtype="bfloat16").eval()  
5  
6 input_features = tokenizer("请给我一段大模型的简短介绍：", return_tensors="pd")  
7 outputs = model.generate(**input_features, max_new_tokens=128)  
8  
9 print(tokenizer.batch_decode(outputs[0], skip_special_tokens=True))
```

transformers 文本生成代码参考：

```
</> Bash |  
  
1 from transformers import AutoModelForCausalLM, AutoTokenizer  
2  
3 model_name = "Qwen/Qwen3-4B-Instruct-2507"  
4  
5 # load the tokenizer and the model  
6 tokenizer = AutoTokenizer.from_pretrained(model_name)  
7 model = AutoModelForCausalLM.from_pretrained(  
8     model_name,  
9     torch_dtype="auto",  
10    device_map="auto"  
11 )  
12  
13 # prepare the model input  
14 prompt = "Give me a short introduction to large language model."  
15 messages = [  
16     {"role": "user", "content": prompt}  
17 ]  
18 text = tokenizer.apply_chat_template(  
19     messages,  
20     tokenize=False,  
21     add_generation_prompt=True,  
22 )  
23 model_inputs = tokenizer([text], return_tensors="pt").to(model.device)  
24  
25 # conduct text completion  
26 generated_ids = model.generate(  
27     **model_inputs,  
28     max_new_tokens=16384  
29 )  
30 output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()  
31  
32 content = tokenizer.decode(output_ids, skip_special_tokens=True)  
33  
34 print("content:", content)  
35
```

## 2.7 模型训练验证

PaddleFormers 训练参考：<https://github.com/PaddlePaddle/PaddleFormers/blob/release/v1.0/examples/README.md>

ms-swift 训练参考：[https://github.com/modelscope/ms-swift/blob/main/README\\_CN.md](https://github.com/modelscope/ms-swift/blob/main/README_CN.md)

## 2.8 模型单测

在完成组网构建后，我们需要在tests.transformers下新增ernie4\_5目录，新增test\_modeling文件。对于一个大模型组网，需要添加的单测包括以下几个类

- 模型组网单测类：定义class Ernie4\_5ModelTest
- 模型下载单测类：定义class Ernie4\_5ModelIntegrationTest

需要使用到的导包

```
• from tests.testing_utils import slow （slow 装饰器标识为慢单测）
• from tests.transformers.test_configuration_common import ConfigTester （用于测试Config相关公用类）
• from tests.transformers.test_generation_utils import GenerationTesterMixin （用于测试模型自回归生成的工具类）
• from tests.transformers.test_modeling_common import (
    ModelTesterMixin, （用于模型基本功能的工具类）
    ModelTesterPretrainedMixin, （用于模型下载的工具类）
    ids_tensor, （创建数值位于词表大小的张量）
    random_attention_mask, （随机生成attention_mask）
)
```

- 1. 新增定义Ernie4\_5ModelTester，将测试Ernie4\_5模型需要用到的通用方法（例如创建模型输入的功能）统一放在一个类中
- 2. 定义Ernie4\_5ModelTest(ModelTesterMixin, GenerationTesterMixin, unittest.TestCase):
  - a. 这个类主要是测试模型的基本功能，能否按要求进行定义
  - b. 能否正常进行单次forward、自回归推理

|    |                                 |                                                                                     |
|----|---------------------------------|-------------------------------------------------------------------------------------|
| 1  | 单测方法                            | 说明                                                                                  |
| 2  | test_model                      | 使用self.model_tester.prepare_config_and_inputs()构建输入，并验证Ernie4_5Model模型forward输出是否正确 |
| 3  | test_model_attention_mask       | 检查在不同的attention_mask形状上的输出是否具有一样的logits输出                                           |
| 4  | test_model_position_ids         | 检查Llama模型在没有显式提供position_ids和显式提供position_ids时的行为一致性                                |
| 5  | test_generate_without_input_ids | （暂不考虑）                                                                              |
| 6  | test_ernie4_5_lm_head_model     | 验证Ernie4_5ForCausalLM的输出形状和类型是否符合预期                                                 |
| 7  | test_ernie4_5_gqa_model         | 验证GQA情况下Ernie4_5ForCausalLM的输出形状和类型是否符合预期                                           |
| 8  | test_attention_outputs          | 验证模型在请求输出注意力权重时，能够正确返回所有层的注意力矩阵，并且这些矩阵的形状符合预期。                                      |
| 9  | test_beam_search_generate       | 验证模型使用 generate() 方法和直接调用 beam_search() 方法产生的结果是否一致，确保生成算法的正确实现。                    |
| 10 | test_greedy_generate            | 验证模型使用 generate() 方法和直接调用 greedy_search() 方法产生的结果是否一致，确保贪婪搜索算法的正确实现。                |
| 11 | test_group_beam_search_generate | 验证模型使用 generate() 方法和直接调用 group_beam_search() 方法产生的结果是否一致，确保分组束搜索（多样化束搜索）算法的正确实现。   |
| 12 | test_resize_tokens_embeddings   | 验证模型能够正确调整词汇表大小，包括扩大和缩小词汇表，同时确保前向传播仍然正常工作，并且原有词汇的嵌入向量保持不变。                          |
| 13 | test_sample_generate            | 验证模型使用 generate() 方法和直接调用 sample() 方法产生的结果是否一致，确保随机采样生成算法的正确实现。                     |
| 14 | test_determinism                | 验证模型在相同输入下多次运行是否产生完全相同的输出，确保模型计算具有确定性和可重现性。                                         |
| 15 | test_model_name_list            | （暂时不用考虑）                                                                            |
| 16 | test_save_load                  | 验证模型能够正确保存到磁盘并从磁盘加载，且加载后的模型与原始模型产生相同的输出。                                            |
| 17 | test_hidden_states_output       | 验证模型在请求输出隐藏状态时，能够正确返回所有层的隐藏状态，并且这些状态的形状和数量符合预期。                                     |

表3 Ernie4\_5ModelTest 需要完善的单测列表

- 3. Ernie4\_5ModelIntegrationTest(ModelTesterPretrainedMixin, unittest.TestCase):
  - a. 定义类变量 base\_model\_class = Ernie4\_5Model，让工具类ModelTesterPretrainedMixin能够执行下载和加载的单测
- 4. 如何新增一个random权重，上传权重到Alstudio 找 @黄志恒 @王彦博



```
1 from transformers import Ernie4_5Config, Ernie4_5ForCausalLM
2 config = Ernie4_5Config(hidden_size=16, num_hidden_layers=1, num_attention_heads=2)
3 model = Ernie4_5ForCausalLM.from_config(config)
4 model.save_pretrained("tiny-random-ernie45")
```

5. 如何修改from\_pretrained下载源

</> Plain Text |

```
1 默认下载源是hf，可以通过下面任意一种方式控制下载源
2 1.export DOWNLOAD_SOURCE="huggingface"/"aistudio"/"modelscope"
3 2. 这种方式优先级更高tokenizer = AutoTokenizer.from_pretrained("deepseek-ai/DeepSeek-V3.1-Base",
  download_hub="huggingface"/"aistudio"/"modelscope")
```

2.9 模型template验证

验证流程：

训练时打开FLAGS\_enable\_dataset\_debug=true，会打印如下所示的log：

```
[dataset debug] Debug mode enabled
[dataset debug] input: <|begin_of_sentence|>User: 列举三个人工智能应用程序
Assistant: 人工智能的三个应用程序是自然语言处理、计算机视觉和人脸识别。
[dataset debug] labels: User: 列举三个人工智能应用程序
Assistant: 人工智能的三个应用程序是自然语言处理、计算机视觉和人脸识别。</s>

[dataset debug] Debug mode enabled
[dataset debug] input: <|begin_of_sentence|>User: 写一篇关于电动汽车的好处的博客文章。
Assistant: 电动汽车正在改变我们行驶的方式。电动汽车不仅可以减少空气污染和温室气体排放，而且还高效、经济实惠，并且需要比传统汽车更少的维护。使用电动汽车，驾驶员可以享受比汽油动力车更安静、更平稳的行驶体验。此外，电动汽车有潜力在长期内为驾驶员节省大量费用，因为他们不需要加油或换油。总体而言，电动汽车正在越来越成为全球驾驶员的一种有吸引力的选择。
[dataset debug] labels: User: 写一篇关于电动汽车的好处的博客文章。
Assistant: 电动汽车正在改变我们行驶的方式。电动汽车不仅可以减少空气污染和温室气体排放，而且还高效、经济实惠，并且需要比传统汽车更少的维护。使用电动汽车，驾驶员可以享受比汽油动力车更安静、更平稳的行驶体验。此外，电动汽车有潜力在长期内为驾驶员节省大量费用，因为他们不需要加油或换油。总体而言，电动汽车正在越来越成为全球驾驶员的一种有吸引力的选择。</s>

[dataset debug] Debug mode enabled
[dataset debug] input: <|begin_of_sentence|>User: 使用以下单词创作一个简短的故事：神话传说，草药，海洋
Assistant: 从前，有一件神话传说中的事情发生在海洋中。据说，在海洋中生长着一种特殊的草药，如果你摘下它，就能实现愿望。一天，一名勇敢的探险家决定承担这个艰巨的任务，航行进入未知的深海，寻找这种神奇的草药。
[dataset debug] labels: User: 使用以下单词创作一个简短的故事：神话传说，草药，海洋
Assistant: 从前，有一件神话传说中的事情发生在海洋中。据说，在海洋中生长着一种特殊的草药，如果你摘下它，就能实现愿望。一天，一名勇敢的探险家决定承担这个艰巨的任务，航行进入未知的深海，寻找这种神奇的草药。</s>

[dataset debug] Debug mode enabled
[dataset debug] input: <|begin_of_sentence|>User: 生成一句与谚语“及时行乐”的谚语相关的例子。
Assistant: 拖延是时间的小偷-立即行动，才能在以后节省时间和精力。
[dataset debug] labels: User: 生成一句与谚语“及时行乐”的谚语相关的例子。
Assistant: 拖延是时间的小偷-立即行动，才能在以后节省时间和精力。</s>
```

查看打印的input和labels是否符合预期来确认数据流是否正常， 需要关注以下几点：

- 1. 原始数据是否被误删的部分
- 2. label是否以eos token结尾